

Software Development Plan

Real-Time AR Soil/Crop Analysis System

Evolution from Pixel Hunt Challenge to AR-Based Analysis

1. Executive Summary

Project Name: Real-Time AR Soil/Crop Analysis System
Base Technology: Pixel Hunt Challenge (Mouse Hover → Camera AR)
Core Framework: Babylon.js with WebXR
Duration: 10 weeks (2.5 sprints)
Team Size: 5-6 developers

1.1 Project Evolution

Pixel Hunt (Current)	→	AR Analysis (Target)
Mouse Hover on Map	→	Camera View of Ground
Static Pixel Data	→	Real-time Analysis
2D Map Interface	→	3D AR Overlay
Pre-calculated Values	→	Live AI Processing

1.2 Data Format Consistency

Maintained from Pixel Hunt:

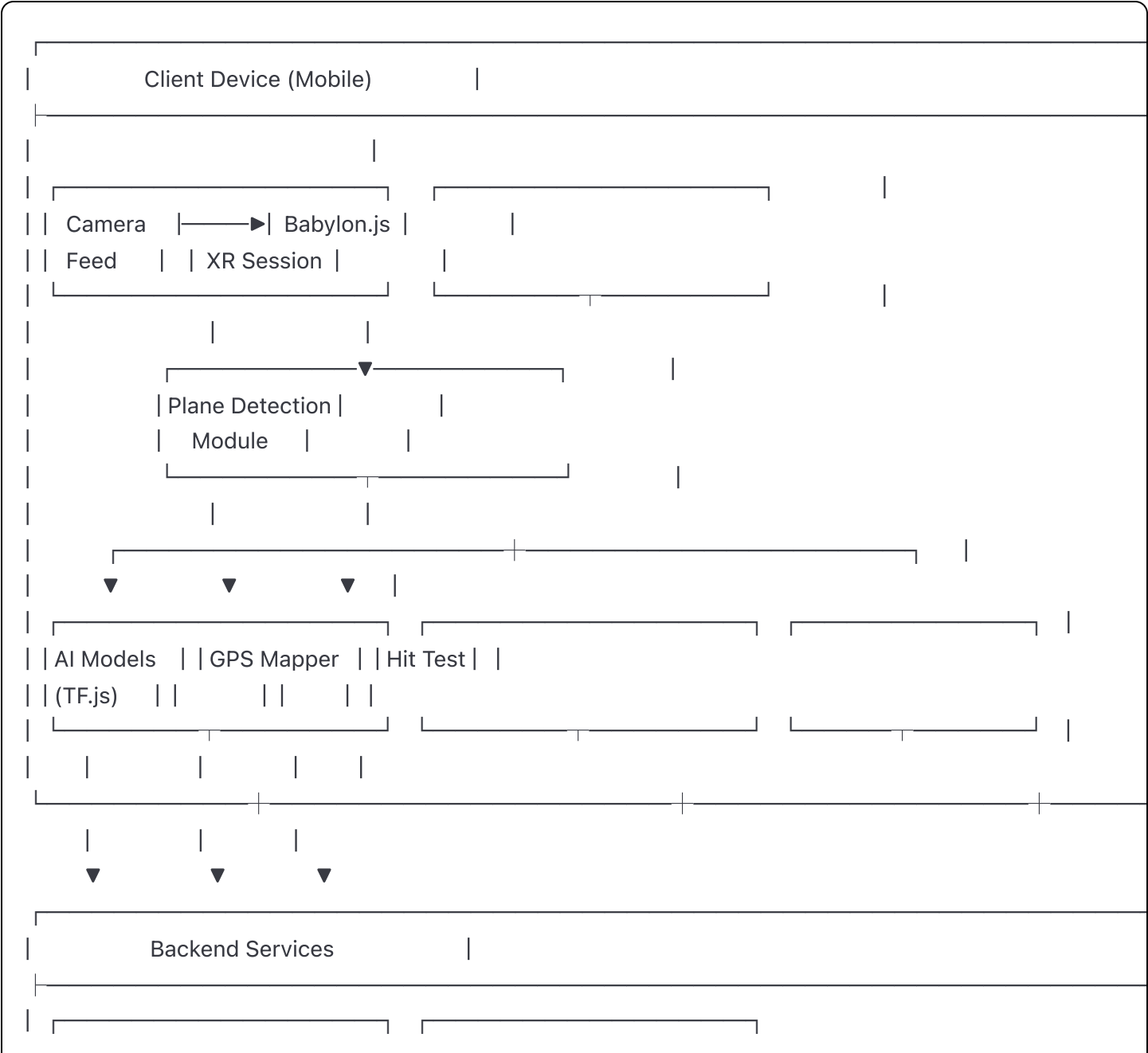
```
javascript
// Same data structure, different input source
{
  pixel: [x, y],    // Grid position
  moisture: 24.8,   // Soil moisture %
  ndvi: 0.433,     // Vegetation index
  temperature: 21.3, // Surface temp °C
  type: "pasture",  // Land classification
  health: 80,       // Health score %
  index: 81         // Grid index
}
```

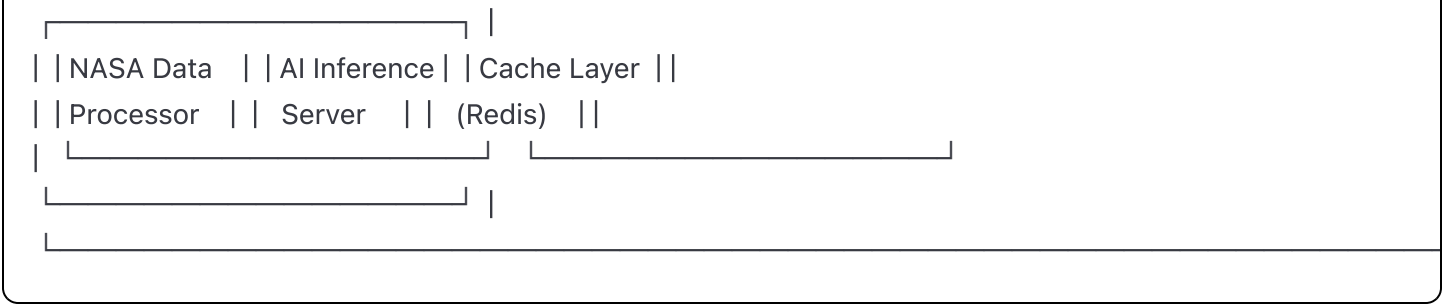
2. Technical Architecture

2.1 Core Technology Stack

Component	Technology	Purpose	Change from Original
3D Engine	Babylon.js 6.x	AR rendering	Replaced Three.js
AR Framework	Babylon.js XR	WebXR implementation	Native Babylon support
AI Models	TensorFlow.js	Object classification	New addition
Plane Detection	WebXR Plane API	Ground detection	Enhanced from basic AR
Data Pipeline	NASA API + Cache	Satellite data	Same as Pixel Hunt
GPS Mapping	Geolocation API	Position tracking	Real-time updates

2.2 System Architecture Diagram





3. Development Phases

Phase 1: Foundation (Weeks 1-3)

Goal: Babylon.js AR Setup with Plane Detection

Technical Tasks:

1. Babylon.js WebXR Integration

- Setup Babylon.js scene
- Initialize WebXR session
- Configure AR camera
- Implement plane detection

2. Plane Detection System

```
javascript

// Babylon.js Plane Detection
const xr = await scene.createDefaultXRExperienceAsync({
  uiOptions: {
    sessionMode: "immersive-ar",
    referenceSpaceType: "local-floor"
  },
  optionalFeatures: true
});

const featureManager = xr.baseExperience.featuresManager;
const planeDetection = featureManager.enableFeature(
  BABYLON.WebXRFeatureName.PLANE_DETECTION,
  "latest",
  { preferredDetectorOptions: { preferredPolygonType: "polygon" } }
);
```

3. Ground vs Sky Classification

- Implement normal vector analysis
- Horizontal plane validation
- Sky detection algorithm

Deliverables:

- Working AR camera in Babylon.js
- Plane detection for ground surfaces
- Basic hit testing implementation

Phase 2: AI Integration (Weeks 4-6)

Goal: Object Classification and Analysis Models

AI Model Stack:

```
javascript

// Model Architecture
models: {
  terrainClassifier: {
    input: [224, 224, 3], // Camera frame
    output: ['soil', 'grass', 'crop', 'concrete', 'water'],
    accuracy: 0.92
  },
  cropIdentifier: {
    input: [224, 224, 3],
    output: ['corn', 'wheat', 'soybean', 'pasture', 'bare'],
    accuracy: 0.88
  },
  healthAnalyzer: {
    input: [224, 224, 3],
    output: health_score (0-100),
    accuracy: 0.85
  }
}
```

Implementation Tasks:

1. TensorFlow.js Integration

- Load pre-trained models
- Real-time inference pipeline

- Frame extraction from video

2. Classification Pipeline

javascript

```
async function analyzeFrame(videoFrame) {  
  // Extract frame from camera  
  const tensor = tf.browser.fromPixels(videoFrame);  
  
  // Preprocess  
  const preprocessed = tf.image.resizeBilinear(tensor, [224, 224]);  
  const normalized = preprocessed.div(255.0);  
  
  // Run models in parallel  
  const [terrain, crop, health] = await Promise.all([  
    terrainModel.predict(normalized),  
    cropModel.predict(normalized),  
    healthModel.predict(normalized)  
  ]);  
  
  return {  
    type: getTopClass(terrain),  
    crop: getTopClass(crop),  
    health: health.dataSync()[0] * 100  
  };  
}
```

Deliverables:

- Trained TensorFlow.js models
- Real-time classification system
- 30 FPS analysis performance

Phase 3: Data Integration (Weeks 7-8)

Goal: NASA Data Mapping to Camera Position

GPS Coordinate Mapping:

javascript

```

// Convert AR hit point to GPS coordinates
function hitPointToGPS(hitResult, deviceLocation) {
  const hitMatrix = hitResult.transformationMatrix;
  const position = BABYLON.Vector3.TransformCoordinates(
    BABYLON.Vector3.Zero(),
    hitMatrix
  );

  // Calculate offset from device
  const offsetMeters = {
    north: position.z,
    east: position.x
  };

  // Convert to GPS
  const gps = {
    lat: deviceLocation.lat + (offsetMeters.north / 111111),
    lon: deviceLocation.lon + (offsetMeters.east / (111111 * Math.cos(deviceLocation.lat * Math.PI / 180)))
  };

  return gps;
}

```

Pixel Hunt Data Format Integration:

```

javascript

// Maintain exact same data structure
function formatAnalysisData(gps, aiResults, nasaData) {
  return {
    pixel: gridPosition(gps), // Convert GPS to grid
    moisture: nasaData.soil_moisture,
    ndvi: nasaData.ndvi,
    temperature: nasaData.surface_temp,
    type: aiResults.terrain, // From AI model
    health: aiResults.health, // From AI model
    index: calculateIndex(gps)
  };
}

```

Deliverables:

- GPS coordinate mapping system

- NASA API integration
- Real-time data updates
- Pixel Hunt format compatibility

Phase 4: Real-time Visualization (Weeks 9-10)

Goal: AR Overlay with Live Data Updates

Babylon.js AR Overlay:

```
javascript
```

```

// Create AR data overlay
class ARDataOverlay {
  constructor(scene) {
    this.scene = scene;
    this.gui = BABYLON.GUI.AdvancedDynamicTexture.CreateFullscreenUI("UI");
    this.dataPanel = null;
  }

  updateDisplay(data) {
    // Create/update floating panel at hit point
    if (!this.dataPanel) {
      this.dataPanel = new BABYLON.GUI.Rectangle();
      this.dataPanel.width = "200px";
      this.dataPanel.height = "150px";
      this.dataPanel.cornerRadius = 10;
      this.dataPanel.background = "rgba(0,0,0,0.8)";
      this.gui.addControl(this.dataPanel);
    }

    // Update text - exact Pixel Hunt format
    const text = new BABYLON.GUI.TextBlock();
    text.text = `
      Pixel [${data.pixel[0]}, ${data.pixel[1]}]
      Moisture: ${data.moisture}%
      NDVI: ${data.ndvi}
      Temp: ${data.temperature}°C
      Type: ${data.type}
      Health: ${data.health}%
      Index: ${data.index}
    `;
    text.color = "white";
    text.fontSize = 14;

    this.dataPanel.clearControls();
    this.dataPanel.addControl(text);
  }

  positionAtHitPoint(hitResult) {
    // Position overlay at 3D hit point
    const worldPos = hitResult.pickedPoint;
    const screenPos = BABYLON.Vector3.Project(
      worldPos,
      BABYLON.Matrix.Identity(),

```



```

        this.scene.getTransformMatrix(),
        this.scene.activeCamera.viewport.toGlobal(
            this.scene.getEngine().getRenderWidth(),
            this.scene.getEngine().getRenderHeight()
        )
    );

    this.dataPanel.leftInPixels = screenPos.x - 100;
    this.dataPanel.topInPixels = screenPos.y - 75;
}
}
```

Deliverables:

- Real-time AR overlay
- 60 FPS rendering performance
- Smooth data updates
- Mobile-optimized interface

4. Sprint Planning

Sprint 1: AR Foundation (Weeks 1-4)

Objective: Complete Babylon.js AR setup with working plane detection

Task	Story Points	Priority
Babylon.js scene setup	5	P0
WebXR initialization	8	P0
Plane detection implementation	13	P0
Hit testing system	8	P0
Ground validation logic	5	P1
Performance optimization	3	P2

Success Metrics:

- AR session starts in <2 seconds
- Plane detection works on 95% of surfaces
- 30+ FPS maintained

Sprint 2: Intelligence Layer (Weeks 5-8)

Objective: AI models integrated with real-time analysis

Task	Story Points	Priority
TensorFlow.js setup	3	P0
Model loading system	5	P0
Frame extraction pipeline	8	P0
Classification implementation	8	P0
GPS coordinate mapping	5	P1
NASA data integration	8	P1

Success Metrics:

- Model inference <100ms
- 85%+ classification accuracy
- Real-time data updates

Sprint 3: Polish & Optimization (Weeks 9-10)

Objective: Production-ready system with smooth UX

Task	Story Points	Priority
AR overlay UI	8	P0
Data visualization	5	P0
Performance tuning	5	P1
Error handling	3	P1
Testing & debugging	8	P0

5. Technical Requirements

5.1 Performance Targets

Metric	Target	Critical
AR Session Start	<2s	<3s
Plane Detection	<1s	<2s
Model Inference	<100ms	<150ms
Data Update Rate	10Hz	5Hz

Metric	Target	Critical
Frame Rate	30 FPS	25 FPS
Battery Life	2hr continuous	1hr minimum

5.2 Device Requirements

Minimum Specifications:

- **iOS:** iPhone XS or newer, iOS 14+
- **Android:** ARCore compatible, Android 8+
- **RAM:** 3GB minimum
- **Camera:** Rear camera with autofocus
- **Sensors:** Gyroscope, Accelerometer, Magnetometer

5.3 Browser Support

- Chrome 89+ (Android)
- Safari 14.5+ (iOS)
- Edge 89+ (Android)

6. Risk Management

Technical Risks

Risk	Probability	Impact	Mitigation
Babylon.js AR compatibility	Medium	High	Fallback to camera-only mode
Model accuracy outdoors	High	Medium	Multiple model ensemble
GPS accuracy	Medium	Medium	Visual marker assistance
Network latency	Low	High	Aggressive caching
Battery drain	High	Medium	Quality settings adjustment

Mitigation Strategies

1. Progressive Enhancement

```
javascript
```

```
// Fallback chain
if (WebXRSupported) {
  // Full AR experience
} else if (CameraSupported) {
  // Camera with overlay
} else {
  // Static analysis mode
}
```

2. Model Optimization

- Quantization to reduce size
- Edge TPU acceleration
- Progressive model loading

3. Caching Strategy

- IndexedDB for models
- Service Worker for data
- Coordinate-based cache keys

7. Quality Assurance

7.1 Testing Strategy

Unit Testing:

- Babylon.js scene components
- AI model predictions
- Data transformation functions
- GPS calculations

Integration Testing:

- AR session lifecycle
- Model pipeline
- Data flow end-to-end

Field Testing:

- Various lighting conditions

- Different terrain types
- Network conditions
- Device performance

7.2 Success Criteria

- ☐ Plane detection success rate >95%
 - ☐ Classification accuracy >85%
 - ☐ Data update latency <200ms
 - ☐ No memory leaks over 30min usage
 - ☐ Battery usage <15% per 10min
-

8. Development Tools

Required Software

```
json
{
  "development": {
    "babylon.js": "^6.0.0",
    "tensorflow.js": "^4.2.0",
    "typescript": "^5.0.0",
    "webpack": "^5.75.0",
    "babel": "^7.20.0"
  },
  "testing": {
    "jest": "^29.0.0",
    "puppeteer": "^19.0.0",
    "lighthouse": "^10.0.0"
  },
  "deployment": {
    "docker": "^20.10.0",
    "kubernetes": "^1.25.0",
    "nginx": "^1.23.0"
  }
}
```

9. Team Structure

Role	Responsibilities	Allocation
AR Developer	Babylon.js, WebXR, Plane Detection	100%
ML Engineer	TensorFlow.js, Model Training	100%
Backend Developer	NASA API, Data Pipeline	80%
Frontend Developer	UI/UX, Overlays	60%
DevOps	Deployment, Monitoring	40%

10. Budget Estimation

Category	Cost	Notes
Development (10 weeks)	\$80,000	5 developers
AI Model Training	\$5,000	GPU cloud costs
Infrastructure	\$3,000	AWS/GCP services
Testing Devices	\$2,000	Various phones
Total	\$90,000	

Appendices

Appendix A: Babylon.js AR Code Structure

```
src/
├── ar/
│   ├── BabylonScene.ts
│   ├── PlaneDetector.ts
│   ├── HitTester.ts
│   └── XRManager.ts
├── ai/
│   ├── ModelLoader.ts
│   ├── Classifier.ts
│   └── HealthAnalyzer.ts
├── data/
│   ├── GPSTMapper.ts
│   ├── NASAClient.ts
│   └── DataFormatter.ts
└── ui/
```

Appendix B: Model Training Pipeline

- Dataset: 50,000 labeled terrain images
- Architecture: MobileNetV3 (optimized for mobile)
- Training: Transfer learning from ImageNet
- Optimization: TensorFlow Lite conversion

Appendix C: Data Flow Sequence

1. Camera frame capture (30 FPS)
2. Plane detection (10 Hz)
3. Hit testing on tap/center
4. AI classification (100ms)
5. GPS calculation
6. NASA data fetch/cache
7. Format to Pixel Hunt structure
8. Display overlay update